DEPARTMENT OF INDUSTRIAL MANAGEMENT

FACULTY OF SCIENCE | UNIVERSITY OF KELANIYA

Mobile UI Design Patterns, Navigation & Material Components

Android Fundamentals — UI Design Patterns

Part 1 — Foundations: Views, Layouts & Events

Part 2 — Moving Between Screens: Intents & Data

Part 3 — Material Design & Responsive Layout

Part 4 — Structuring App-Wide Navigation

Part 5 — Displaying Data at Scale: Lists

Sammani Maddumahewage · Lecturer (Probationary) · BSc (Hons) in Information Technology



Views & Layouts: Building the Login Screen

The Building Blocks

Layouts

Containers that control how everything inside them is positioned.

- LinearLayout, ConstraintLayout

Views

The interactive components the user actually sees and touches.

- TextView, EditText, Button

Every screen we build today starts the same way: a layout in XML, holding views. We'll use this Login screen as our running example for the rest of the lecture.

The Login Screen in XML

```
<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="24dp">

  <EditText
    android:id="@+id/etUsername"
    android:hint="Username" />

  <EditText
    android:id="@+id/etPassword"
    android:hint="Password"
    android:inputType="textPassword" />

  <Button
    android:id="@+id/btnLogin"
    android:text="Login" />
</LinearLayout>
```

Making the UI Interactive: Event Handling

How Android Responds to the User

The XML we just wrote is silent — tapping Login does nothing until we attach Kotlin code to it.

Step 1 — Find the view:

```
val username = findViewById<EditText>(R.id.etUsername)
val password = findViewById<EditText>(R.id.etPassword)
val loginButton = findViewById<Button>(R.id.btnLogin)
```

Step 2 — Every event follows the same flow:

User Action → Event Generated → Listener Activated → Code Executed → UI Feedback

Without this pattern, the UI is static and the app is non-interactive. It's the single idea behind every button, menu, and list item we'll wire up for the rest of the lecture.

Wiring Up the Login Button

```
loginButton.setOnClickListener {
  val user = username.text.toString()
  val pass = password.text.toString()

  if (user.isNotEmpty() && pass.isNotEmpty()) {
    Toast.makeText(this, "Login Successful",
      Toast.LENGTH_SHORT).show()
  } else {
    Toast.makeText(this, "Please enter all fields",
      Toast.LENGTH_SHORT).show()
  }
}
```

Navigating With Explicit Intents

What Is an Intent?

A validated login is only useful if it takes the user somewhere. An Intent is the messaging object that makes that jump happen.

An explicit Intent names both:

- The current Activity (context) — this
- The target Activity class — HomeActivity::class.java

The same pattern links Login to a Registration screen, and back again:

```
// Registration -> Login
tvLoginLink.setOnClickListener { finish() }
```

From Login to Home

```
// Continues the Login button's click listener
if (isValidUser) {
  val intent = Intent(this, HomeActivity::class.java)
  startActivity(intent)
}

// Login -> Registration
btnGoToRegister.setOnClickListener {
  val intent = Intent(this, RegisterActivity::class.java)
  startActivity(intent)
}
```

Passing & Receiving Data Between Activities

Sending Data on the Way Out

```
// Just before startActivity(intent) on the Login screen
intent.putExtra("USERNAME", username)
startActivity(intent)
```

putExtra() attaches key–value data to the Intent — here, the username the person just logged in with.

This is the same Intent from the previous slide; we're simply adding a payload to it before it launches HomeActivity.

Reading Data on the Way In

```
// Inside HomeActivity.onCreate()
val username = intent.getStringExtra("USERNAME")
tvWelcome.text = "Welcome $username"
```

getStringExtra() reads the value back out by the same key used in putExtra().

This send-and-receive pattern enables:

- Personalization of the next screen
- Context sharing between Activities
- Dynamic UI updates driven by real data

Material Design: The Principles Behind Android's UI

Three Foundational Principles

1. Material is Metaphorical

Components behave like physical objects — buttons look pressable, cards look elevated.

2. Bold, Intentional Design

Clear typography and meaningful color make primary actions and errors stand out.

3. Motion Provides Meaning

Ripples and transitions — like the one between our Login and Registration screens — guide attention and confirm cause and effect.

Why a Plain EditText Isn't Enough

```
<EditText
  android:hint="Email" />
```

A traditional EditText:

- Has no built-in error handling
- Looks plain, with no visual feedback

Material Design introduces TextInputLayout, which wraps EditText and applies all three principles above at once — that's what we build next.

Material Components in Practice

Email Field

```
<com.google.android.material.textfield.
  TextInputLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:hint="Email Address">

  <com.google.android.material.textfield.
    TextInputEditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textEmailAddress"/>

</com.google.android.material.textfield.
  TextInputLayout>
```

Password Field, With a Visibility Toggle

```
<com.google.android.material.textfield.
  TextInputLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  app:endIconMode="password_toggle"
  android:hint="Password">

  <com.google.android.material.textfield.
    TextInputEditText
    android:id="@+id/etPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textPassword"/>

</com.google.android.material.textfield.
  TextInputLayout>
```

app:endIconMode="password_toggle" adds a show/hide eye icon for free — no extra Kotlin code needed.

Designing for Every Screen: Responsive Layout

What Responsive UI Must Do

Every screen we design has to keep working as conditions change:

- Work correctly on every screen size
- Support orientation changes
- Remain accessible at any density

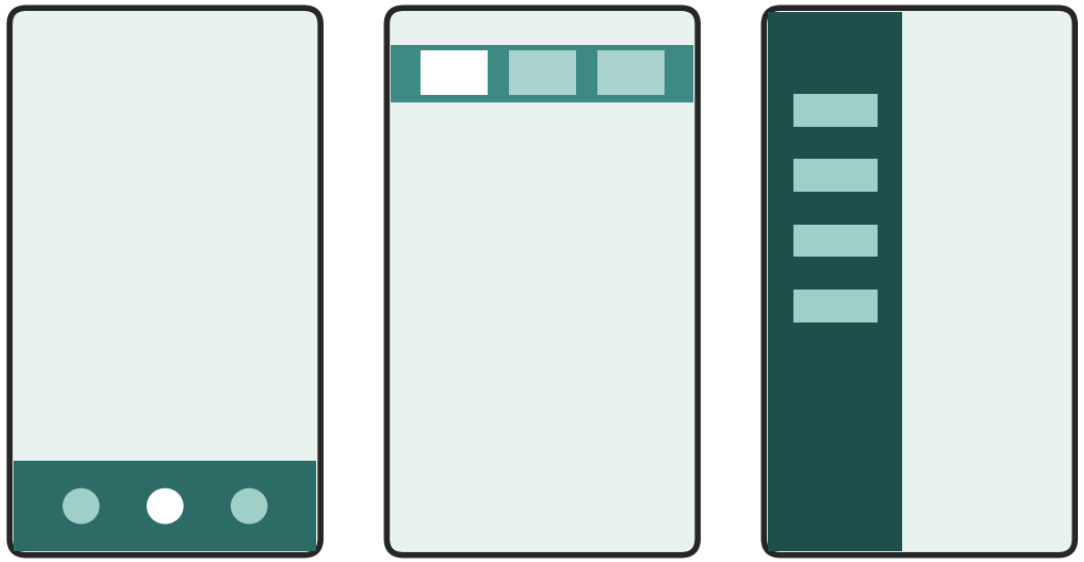
Getting this right is what lets the same Login layout we built in Part 1 look correct on a small phone and a large tablet alike.

Techniques We're Already Using

- Prefer ConstraintLayout over fixed positioning for complex screens
- Avoid fixed pixel dimensions
- Size spacing with dp, and text with sp
- Test on multiple device configurations before shipping

```
android:padding="24dp"      <!-- spacing: dp -->
android:textSize="16sp"     <!-- text: sp -->
```

Choosing an App-Wide Navigation Structure



So far every screen has stood alone. A real app needs a consistent way for users to move between all of its top-level destinations.

- Bottom Navigation — best for 3–5 destinations used frequently
- Tabs — best for switching between related views of the same data
- Navigation Drawer — best for many destinations, including infrequent ones

A Top App Bar (Toolbar) sits above all three — we'll build that first.

Top Navigation: The Toolbar

Toolbar & Menu XML

```
<com.google.android.material.appbar.
  MaterialToolbar
  android:id="@+id/toolbar"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:title="Home"
  app:menu="@menu/top_menu"/>

<!-- res/menu/top_menu.xml -->
<menu xmlns:android="...">
  <item
    android:id="@+id/action_logout"
    android:title="Logout"
    android:showAsAction="always"/>
</menu>
```

Handling Menu Clicks in Kotlin

```
toolbar.setOnMenuItemClickListener {
  if (it.itemId == R.id.action_logout) {
    finish()
    true
  } else false
}
```

The Toolbar:

- Displays the current screen title
- Holds action items, like the Logout button above
- Provides consistent branding across screens

Right now, tapping Logout calls finish() immediately — one accidental tap logs the user out. Let's fix that.

Confirming Actions: Dialogs & Alerts

Guarding the Logout Action

```
// Replaces the bare finish() call from the Toolbar's
// logout handler on the previous slide
AlertDialog.Builder(this)
  .setTitle("Logout")
  .setMessage("Do you want to logout?")
  .setPositiveButton("Yes") { _, _ -> finish() }
  .setNegativeButton("No", null)
  .show()
```

setPositiveButton and setNegativeButton each take a lambda that only runs when that button is tapped. setNegativeButton("No", null) simply dismisses the dialog.

Why Dialogs Matter

Dialogs are used to:

- Confirm a destructive or important action, like logging out
- Display warnings before something irreversible happens
- Show short-lived information without leaving the current screen

Bottom Navigation

XML & Menu

```
<com.google.android.material.
  bottomnavigation.BottomNavigationView
  android:id="@+id/bottomNavigation"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  app:menu="@menu/bottom_menu"/>

<!-- res/menu/bottom_menu.xml -->
<menu xmlns:android="...">
  <item android:id="@+id/nav_home"
    android:title="Home"/>
  <item android:id="@+id/nav_users"
    android:title="Users"/>
</menu>
```

Handling Selection

```
bottomNavigation.setOnItemSelectedListener {
  when (it.itemId) {
    R.id.nav_home -> true
    R.id.nav_users -> true
    else -> false
  }
}
```

Bottom Navigation:

- Gives quick access to top-level destinations
- Designed for 3–5 sections, not more
- The most common pattern in modern Android apps

Navigation Drawer (Left Panel)

DrawerLayout Structure

```
<androidx.drawerlayout.widget.DrawerLayout>

  <FrameLayout
    android:id="@+id/container"/>

  <com.google.android.material.navigation.
    NavigationView
    app:menu="@menu/drawer_menu"
    android:layout_gravity="start"/>

</androidx.drawerlayout.widget.DrawerLayout>
```

When to Use a Drawer

The Navigation Drawer:

- Slides in from the left edge of the screen
- Scales well for apps with many features
- Best suited to larger, feature-rich apps

The container FrameLayout swaps in whichever screen is selected from the drawer's menu. Swapping content in and out of one container, without creating a new Activity, is exactly the problem Fragments are built to solve.

Scaling Beyond Activities: Why Fragments

The Problem With One Activity Per Screen

We now have a Toolbar, Bottom Navigation, and a Drawer — but if every destination is a full Activity, we pay for it:

- Heavy memory usage
- Complex navigation logic between screens
- Duplicated toolbars, drawers, and bottom nav on every screen

A Fragment represents a portion of the UI, not a whole screen, and lives inside an Activity with its own lifecycle.

The Fragment Solution

Fragments solve the problem by:

- Allowing multiple screens to live inside one Activity
- Supporting dynamic UI updates without recreating the Activity
- Working naturally with the drawer and bottom nav we just built

In practice, used for:

- Navigation drawer content panels
- Bottom navigation sections

RecyclerView: Displaying Lists Efficiently

Adapter & ViewHolder

```
class UserAdapter(private val users: List<String>) :
  RecyclerView.Adapter<UserAdapter.UserViewHolder>() {

  class UserViewHolder(view: View) :
      RecyclerView.ViewHolder(view) {
    val name: TextView =
      view.findViewById(R.id.tvUserName)
  }

  override fun onCreateViewHolder(
      parent: ViewGroup, viewType: Int
  ): UserViewHolder {
    val view = LayoutInflater.from(parent.context)
      .inflate(R.layout.item_user, parent, false)
    return UserViewHolder(view)
  }

  override fun onBindViewHolder(
      holder: UserViewHolder, position: Int) {
    holder.name.text = users[position]
  }

  override fun getItemCount(): Int = users.size
}
```

Attaching the RecyclerView

```
recyclerView.layoutManager =
  LinearLayoutManager(this)
recyclerView.adapter = UserAdapter(userList)
```

The Users destination we added to the Bottom Navigation menu needs to show a real, scrollable list — this is how.

Four working parts:

- RecyclerView — the scrolling container in the layout
- Adapter — turns data into rows
- ViewHolder — caches each row's view references
- LayoutManager — arranges rows (linear, grid, ...)

Thank You

Questions?

From a single login screen to a fully navigable, multi-screen app — every piece we built today, in order:

Views & Events → Intents & Data → Material Design → App Navigation → Lists

